Machine Guardian: the single-accelerometer model

From three accelerometers to one: what was built, what is inside it, and how well it works.

28 October 2026 · MAC Attack project · model folder `model/` · generated by `model/report/make_single_accel_report.py` from the saved results

28/28	**4/4**	**8 → 32 → 16 → 1**	**382**
test faults found (FP32 and INT8)	healthy test recordings correct (no false alarm)	network, 833 parameters (E3: 1,281)	accelerator cycles per 1-s window (E3: 566), simulated

> **Read these results with care.** The deployed accelerometer (1) was chosen after its test result had been seen, every healthy recording in the dataset comes from one single motor, and each test split has only 4 healthy and 28 faulty recordings. The numbers show the pipeline works end to end; they do not yet show how it will do on your motor. Section 10 lists the limits.

---

1. Summary

- **Goal.** Turn the existing E3 model (three accelerometers, 22 inputs) into one that needs a single physical accelerometer, as the final Machine Guardian hardware will have, keeping everything else the same so the effect of dropping two sensors is isolated. No remaining-useful-life prediction, no temperature or RPM yet.
- **Built.** Three versions, one per Ottawa accelerometer, each 8 inputs → 32 → 16 → 1, trained exactly like E3 with seeds 42, 43 and 44. The original E3 files were not touched.
- **Chosen.** A rule fixed before training picked **accelerometer 2** on validation, but validation could barely tell the three apart. The team then chose **accelerometer 1**, because it is the only one mounted on the motor itself, like ours. That choice came after the test results, so it is recorded as a post-test decision.
- **Results (accelerometer 1).** Test: 28/28 faults found, no false alarm, in every seed; E3 found 27/28. The INT8 version on the accelerator gives the same 28/28, and the RTL simulation matches Python bit for bit.
- **Health index.** Ranks healthy below faulty perfectly on train and validation (ROC-AUC 1.000) and 0.929 on test; it does not separate the two at 15 Hz, and it is a distance from one healthy motor, not a remaining-life percentage.

---

2. What the model does

Once a second, the system looks at the last second of vibration from one accelerometer and the speed the motor drive is set to. It produces two outputs: **A**, healthy or faulty (from the last 10 seconds), and **B**, a health index from 0% to 100%.

1. Sense
One accelerometer on the motor, 42,000 samples per second (m/s²), plus the drive's commanded frequency f.

→

2. Features (ARM)
Every 1-s window: 7 statistics of the vibration + ln f = 8 numbers.

→

3. Scale to INT8
Subtract the training mean, divide by the training std, round to −127..127.

→

4. Neural net (FPGA)
8 → 32 → 16 → 1 on the 8×8 systolic MAC array: one fault score (logit) per second.

→

5. Decide
Average the last 10 scores. Above the cut → **faulty** (output A).

→

6. Health index (ARM)
Distance of the same 7 statistics from the healthy motor at that speed → 0–100% (output B).

The neural network (steps 3-5) gives output A. The health index (step 6) is a separate statistical calculation on the same numbers, kept apart so it can later take temperature and RPM. Steps 2, 3, 5 and 6 run on the ARM processor; step 4 runs on the team's INT8 systolic MAC array.

3. What is inside it

The 8 inputs, per 1-second window (42,000 samples)

#	Input	What it measures
1	ln(standard deviation)	how strongly the motor vibrates around its average
2	ln(RMS)	overall vibration energy (includes the sensor's constant offset)
3	ln(max |x|)	the largest single spike
4	ln(peak-to-peak)	distance from the lowest to the highest value
5	crest factor = max|x| / RMS	how spiky the signal is compared with its energy (impacts, e.g. bearing damage)
6	skewness	whether the vibration is lopsided (more large swings one way)
7	kurtosis	how heavy the tails are: rare large impacts push it up
8	ln(f)	the speed the drive commands, in Hz (the same motor vibrates more at higher speed)

Exact names and order: `a1_log_std`, `a1_log_rms`, `a1_log_absmax`, `a1_log_p2p`, `a1_crest`, `a1_skew`, `a1_kurt`, `log_f`. Before the network, each input is standardized with the **training** windows' mean and standard deviation only.

The network

Layer	Size	Parameters
Input	8	-
Hidden 1 (ReLU)	32	8×32 + 32 = 288
Hidden 2 (ReLU)	16	32×16 + 16 = 528
Output (logit)	1	16×1 + 1 = 17
Total		**833**

Training (identical to E3)

Setting	Value
Optimizer	Adam, lr 0.001
Loss	BCEWithLogitsLoss, pos_weight 8/56 = 0.143
Batch / epochs	64 / at most 100
Early stopping	patience 15 on validation loss
Seeds	42, 43, 44 (42 exported)
Threshold	from validation (lowest cut with best F1, recall ≥ 0.8)

The data: University of Ottawa electric motor dataset

128 recordings of 10 s at 42 kHz from a SpectraQuest test rig: one healthy motor and seven faulty motors (bearing fault, broken rotor bars, bowed rotor, rotor misalignment, rotor unbalance, stator winding fault, voltage unbalance), each at 8 speed profiles and two loads. Each recording is cut into ten 1-s windows (1,280 windows). The split is by speed, so the model is always tested on speeds it never trained on, and all windows of a recording stay together.

Split	Speeds	Healthy recordings	Faulty recordings
Train	45, 60 Hz; ramps 15→45, 60→30 Hz	8	56
Validation	30 Hz; ramp 45→15 Hz	4	28
Test	15 Hz; ramp 30→60 Hz	4	28

The three accelerometers

Sensor	Model	Where it is mounted
Accelerometer 1 (deployed)	PCB 603C01 (has a temperature sensor)	magnet on the **motor's drive end**
Accelerometer 2	PCB 623C01	stud on the rig's shaft bearing housing near the drive end
Accelerometer 3	PCB 623C01	stud on the rig's shaft bearing housing farthest from the drive end

Source: Sehri & Dumond, Data in Brief (2024), doi:10.1016/j.dib.2024.110144. Single axis, m/s^2, 42,000 samples/s, no filtering.

4. What I did

1. **Read the existing pipeline** (data, features, training, health index, INT8 export, RTL check, reports) and checked the code against the project description: it matched. One detail worth knowing: the 7 statistics come from 8 stored ones (the mean is dropped).
2. **Confirmed E3 still reproduces on this PC.** Retraining E3 gives the same decisions (scores within 0.000007 of the committed ones), and re-exporting it gives identical hex files.
3. **Added the three single-accelerometer models** to `train.py` (`E3_single_accel_a1/a2/a3`) and wrote `single_accel.py` to train them, apply a selection rule written before training, and produce the analysis.
4. **Made the health index work on any accelerometer** (`health.py --accels`), and refitted it on training data only.
5. **Exported to INT8** with the existing method (`export.py`) and generalized its C header.
6. **Ran the exported model on the accelerator's RTL** in Vivado's simulator. This needed a new `--sim vivado` option in `rtl_check.py`, plus a workaround for a Vivado 2019.1 optimizer bug that broke the testbench (not the hardware).
7. **Added a modular feature function** (`features.window_inputs`) that turns one raw 1-s window into the 8 inputs, with a slot to add temperature and RPM features later.
8. **Wrote 24 automated tests** (`test_single_accel.py`) and the reports (`SINGLE_ACCEL_*.md`); fixed two small Windows bugs (report text encoding; an over-strict float check).
9. **Exported accelerometer 1** after the team's mounting decision, and recorded the decision in `weights/E3_single_accel/deployed.json`.

No hardware (RTL) file was changed: the 8-input model fits the existing 8×8 array as is.

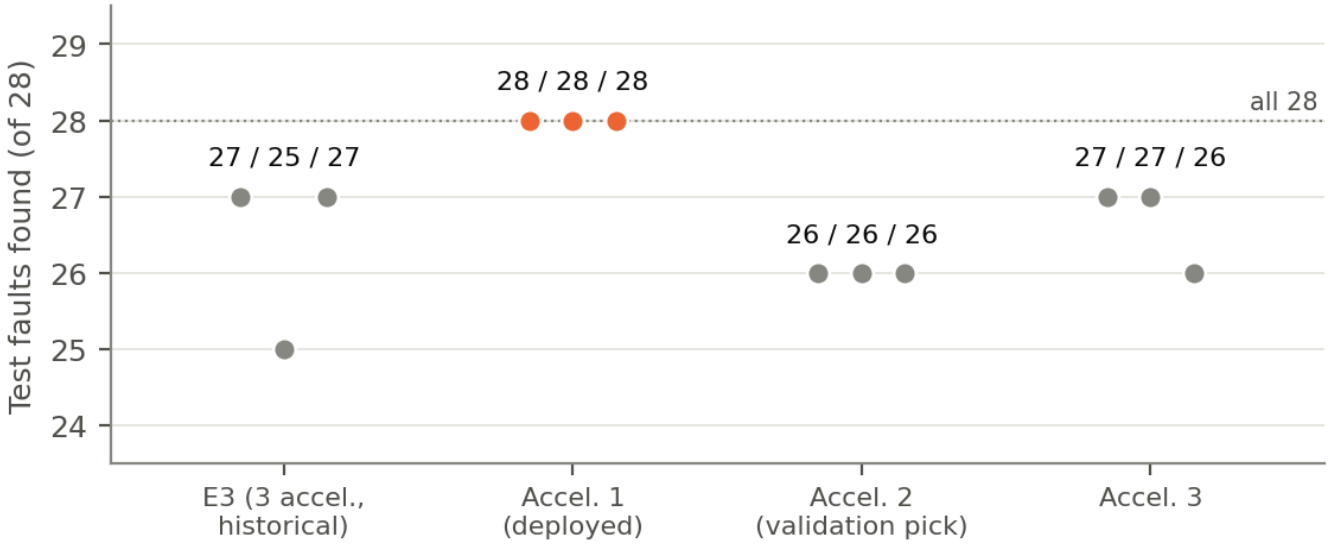
5. Choosing the accelerometer

The rule (fixed before any model was trained) compares the three on **validation only**: recall first, then balanced accuracy, F1, precision, specificity, ROC-AUC, stability across seeds, window-level ROC-AUC, and finally the validation loss.

Model	Val. recall	Val. F1	Val. ROC-AUC	Window ROC-AUC	Val. loss	Rule rank
Accelerometer 1	1.000	1.000	1.000	0.9990	0.0102	3
Accelerometer 2	1.000	1.000	1.000	1.0000	0.0022	1
Accelerometer 3	1.000	1.000	1.000	0.9998	0.0037	2

All three were perfect on validation, so the rule fell through to its last two tie-breakers, where accelerometer 2 was slightly ahead. Those gaps amount to a few of the 320 windows: validation could not really rank the sensors.

Model (seed 42, test)	Inputs	Params	Faults found	Recall	F1	Bal. acc.	MCC	Faults found, seeds 42/43/44
E3, accelerometers 1+2+3 (historical)	22	1,281	27/28	0.964	0.982	0.982	0.878	27 / 25 / 27
Accelerometer 1 (deployed)	8	833	28/28	1.000	1.000	1.000	1.000	28 / 28 / 28
Accelerometer 2 (validation pick)	8	833	26/28	0.929	0.963	0.964	0.787	26 / 26 / 26
Accelerometer 3	8	833	27/28	0.964	0.982	0.982	0.878	27 / 27 / 26



Test faults found out of 28: one dot per seed (42, 43, 44, left to right), counts above. E3 is the historical result.

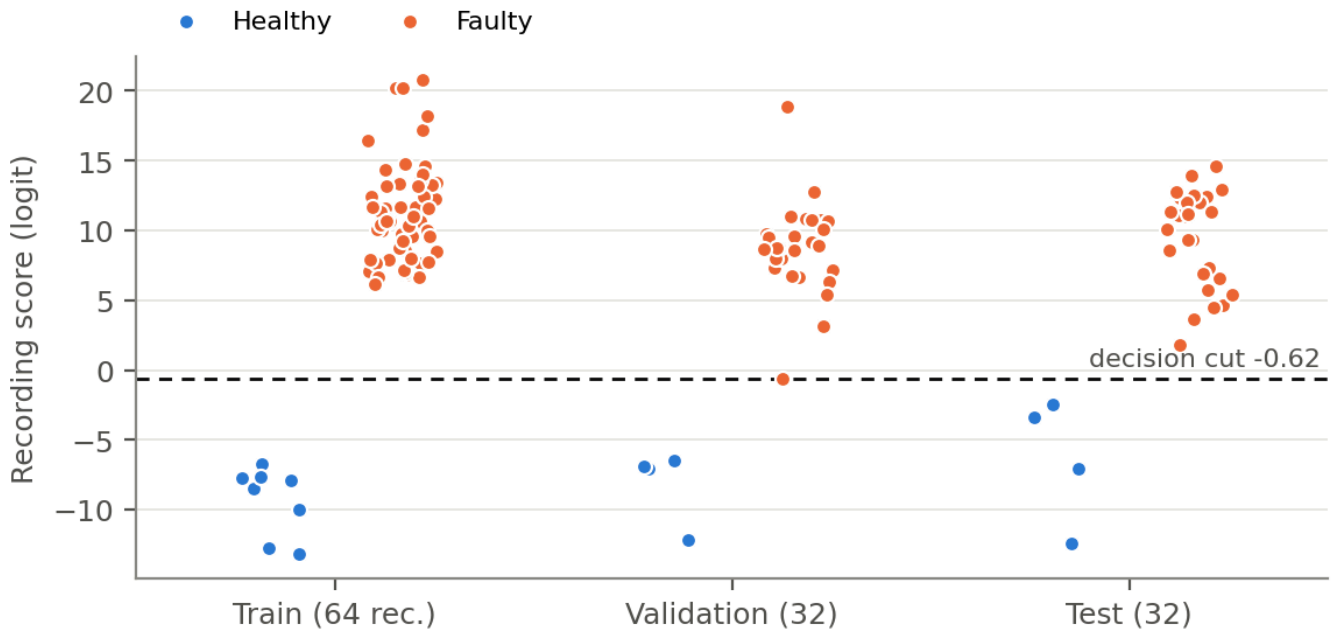
> **Decision.** Accelerometer 1 was chosen for deployment because it is mounted on the motor, as our single accelerometer will be (2 and 3 sit on the rig's bearing housings). It also had the best test result, but that was seen before the decision, so its 28/28 is optimistic. The validation rule's choice (accelerometer 2, 26/28 on test) stays documented in `selected.json`.

6. Results of the deployed model (accelerometer 1)

Healthy / faulty classification, seed 42, FP32 (one decision per 10-s recording)

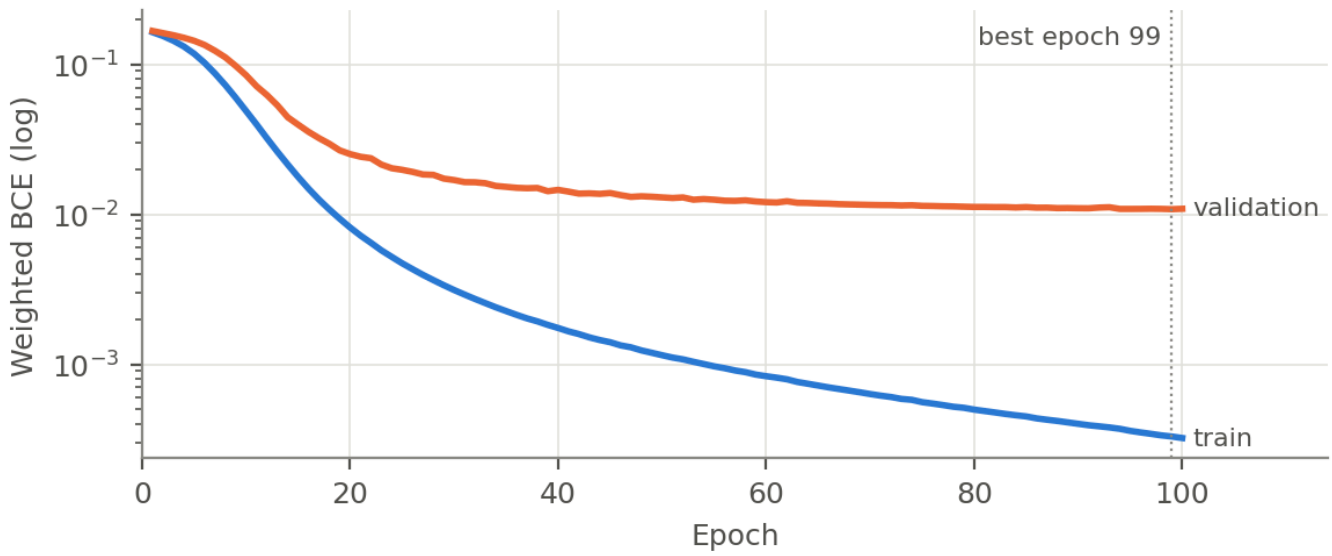
Split	Faults found	Healthy correct	Accuracy	Recall	Precision	F1	Bal. acc.	Specificity	ROC-AUC	MCC
Train	56/56	8/8	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Validation	28/28	4/4	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Test	28/28	4/4	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000

Seeds 43 and 44 give exactly the same on validation and test (28/28, 4/4): no spread between seeds.



Every recording's score (the average of its ten 1-second scores). Above the dashed cut means faulty. The cut was set on validation, exactly at its mildest fault (the orange point on the line). The closest calls are at 15 Hz in the test split: the healthy recordings at −3.4 and −2.5, the faulty S_W_1_0 at 1.8.

Training



Training and validation loss, seed 42. Validation loss was still improving at the end (best epoch 99 of 100): no sign of overfitting within the 100 epochs.

One-second decisions by speed (validation + test, seed 42)

Before the 10-second average, single windows can be wrong. The averaging removes all of these errors.

Drive frequency	Faulty windows missed	Healthy windows flagged
14-20 Hz	0 of 168	1 of 24
20-30 Hz	0 of 42	0 of 6
30-40 Hz	7 of 221	0 of 31
40-50 Hz	0 of 81	0 of 11
50-61 Hz	0 of 48	0 of 8

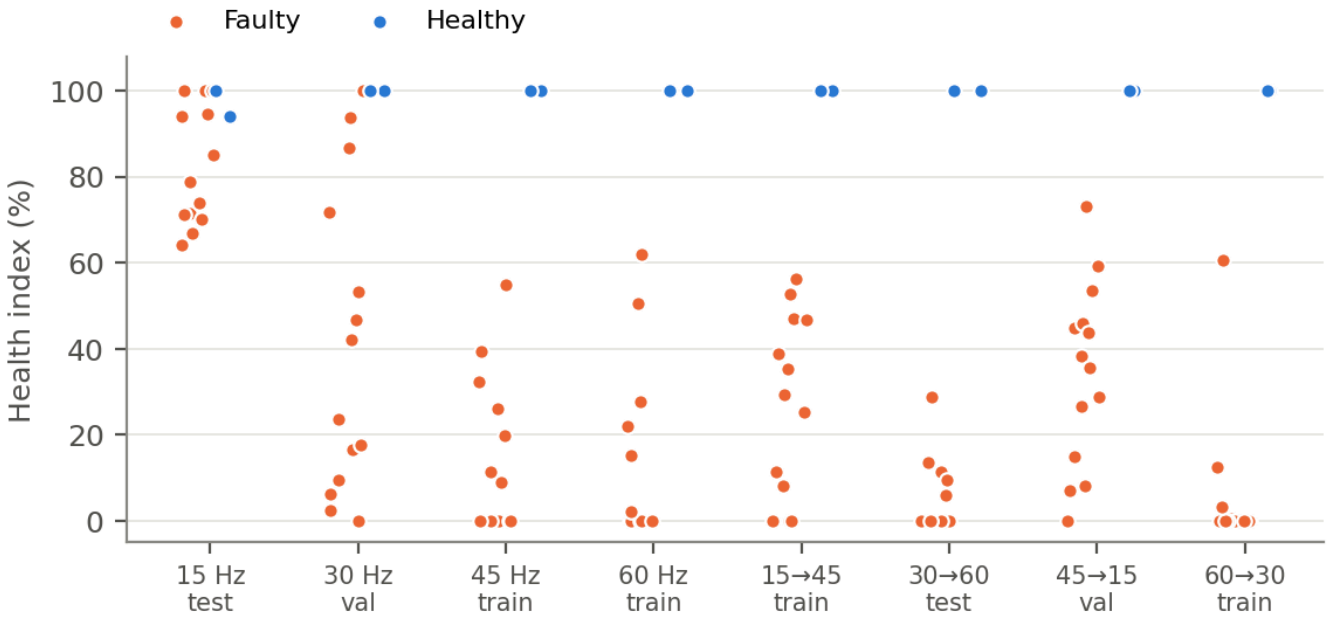
Compared with the original E3

- E3 (three accelerometers, historical, trained on the Mac): 27/28 test faults, recall 0.964, F1 0.982, MCC 0.878; mean recall over seeds 0.940 ± 0.034.
- Accelerometer 1 alone: 28/28 in all seeds, with a third fewer parameters and a third fewer accelerator cycles.
- But its healthy 15-Hz recordings sit much closer to the cut (−3.4 and −2.5, against about −27 and −20 for E3): the margin at low speed is smaller.

7. Health index (output B)

For each of the 7 vibration statistics, a straight line in ln(speed) is fitted to the healthy **training** windows, with the normal spread around it. A window's distance is how many spreads it lies from those lines on average; a recording's distance D is the average of its 10 windows. Two anchors, also from training only, turn D into a percentage: D ≤ 1.92 reads 100%, D ≥ 4.49 (the typical faulty training recording) reads 0%.

Split	Healthy recordings (%)	Faulty median	Faulty reading 100%	Faulty at 0%	ROC-AUC of D
Train	100, 100, 100, 100, 100, 100, 100, 100	0%	0/56	28/56	1.000
Validation	100, 100, 100, 100	37%	1/28	2/28	1.000
Test	94, 100, 100, 100	46%	4/28	9/28	0.929

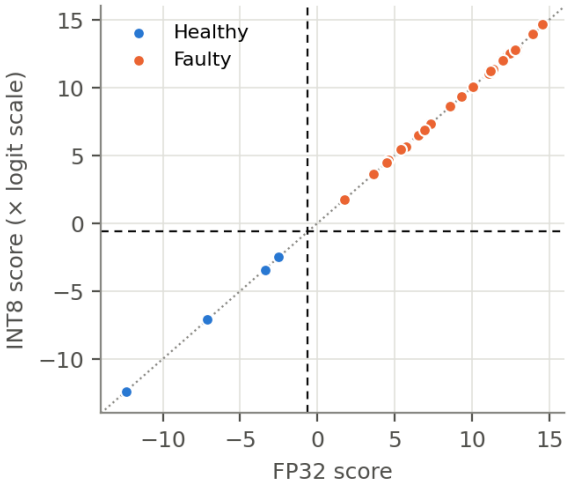


Health index of all 128 recordings by speed profile. Healthy recordings read 100% everywhere except one at 15 Hz (94%). Away from 15 Hz most faults score low; at 15 Hz most faulty recordings also score high.

- **The old 15-Hz problem is gone.** With three accelerometers, the healthy 15-Hz recordings scored 0%. The cause was accelerometer 3: its kurtosis and skewness at 15 Hz were 15-32 spreads away from the healthy line. Without it, they score 94% and 100%.
- **At 15 Hz the index does not separate.** The faulty recording closest to normal (D = 1.50) is nearer the healthy reference than both healthy ones (1.85, 2.07).
- **It is not remaining life.** The dataset has one healthy motor and seven motors with one built-in fault each, with no wear progression, so 0-100% is a distance from normal, not motor strength or lifespan. 4 of 28 faulty test recordings read 100%.

8. INT8 and the accelerator

Item	Value
Layers (K × N)	8×32, 32×16, 16×1
INT8 weights	784 bytes (E3: 1,232)
Passes on the 8×8 array	14 (E3: 22)
Input scale	0.038896 (99.9th percentile, chosen on validation)
Decision threshold	sum of 10 window logits ≥ 10 × (-1308) = -13,080
INT8 test result	28/28 faults, 0/4 false alarms
Decisions changed by INT8	0 in train, validation and test
Average INT8 error per window (test)	0.13 logit
RTL simulation (Vivado xsim)	PASS: every number equals the Python golden model
Cycles, 1 window (M = 1)	382 = 1.78 µs at 214.286 MHz (E3: 566)
Cycles, 10 windows (M = 10)	562 = 2.62 µs (E3: 818)



Test recordings, FP32 score against the INT8 score converted back to logits. Points on the dotted line agree; dashed lines are the cut. No recording changes side.

These are simulation results. Nothing has been measured on the FPGA yet. The 214.286 MHz clock is the earlier on-chip MLP build's; time on the board will also include data transfer and feature calculation.

9. Testing

Check	Result
24 automated tests (`python -m pytest test_single_accel.py`): one accelerometer and 8 inputs per model, in order; training-only scaling; no recording in two splits; all 9 saved models load and reproduce their results; board-side features match the training features; the selection rule; the health index uses the right sensor; INT8 hex files reproduce the test vectors for both exports; the deployment record; E3 files untouched; E3 retrains to the same decisions	24 passed
E3 retrained and re-exported on this PC	same decisions; identical hex files
RTL check, E3 (Vivado xsim)	PASS, 566 cycles (as committed)
RTL check, accelerometer 1 and accelerometer 2 exports	PASS, 382 cycles
Accelerator edge-case suite (`mac.py test --quick` , 2×2 to 16×16 arrays)	PASS, 20/20
`mac.py mlp` on Vivado 2019.1	fails: the same simulator optimizer bug (a tool issue, not the hardware; not caused by this work)
Icarus Verilog runs; PDF report of E3; re-reading the raw dataset	not run: Icarus, reportlab and the 2.5 GB dataset are not on this PC

10. Limitations

- **One healthy motor.** All 16 healthy recordings are the same physical motor; each fault is a different motor. The model may partly be recognising that one quiet motor. An earlier finding: accelerometer 1's spread alone separates that motor from every faulty one.
- **Small, already-seen test set.** 4 healthy and 28 faulty test recordings; 0 false alarms out of 4 is consistent with a real false-alarm rate as high as 60%. E3's development and this choice both used the test results.
- **15 Hz is outside training** (16.5-60 Hz), and the margins are thinnest there.
- **Threshold placement.** The validation rule puts the cut right at the mildest faulty validation recording, which is why accelerometer 2 missed two 15-Hz faults that it ranked correctly.
- **Different motor, different vibration.** Same sensor count does not mean the model will work on our motor: mounting, axis, base and load all change the signal.

11. Before testing the physical three-sensor system

1. Mount our accelerometer like accelerometer 1: on the motor's drive end, same axis, and sample at 42 kHz (or retrain at our rate), keeping the raw m/s^2 signal with its offset.
2. Record a healthy baseline on our own motor at several speeds and loads, synchronized with the temperature and RPM sensors; refit the scaling and the health reference on it, and keep part of it as a truly unseen test set.
3. Run the INT8 model on the ZedBoard (`export/E3_single_accel_a1/`) and measure real cycles and timing.
4. Try a threshold set midway between healthy and faulty scores, as a new pre-declared experiment.
5. Add temperature and measured RPM as new input groups once there is synchronized training data.

Files

Path (in `model/`)	What it is
`single_accel.py`, `test_single_accel.py`	trains and selects the three models; the 24 tests
`train.py`, `features.py`, `health.py`, `export.py`, `rtl_check.py`	existing scripts, extended (E3 behaviour unchanged)
`weights/E3_single_accel_a1/` (and `_a2`, `_a3`)	trained weights per seed and the model description
`weights/E3_single_accel/deployed.json`, `selected.json`	the deployment decision; the validation rule's choice
`export/E3_single_accel_a1/`	INT8 hex files, C header `mg_model.h`, test vectors
`results/E3_single_accel/analysis.md`	every table behind this report
`SINGLE_ACCEL_MODEL_REPORT.md`, `_COMPARISON.md`, `_HARDWARE_REPORT.md`	the detailed reports